

Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Thema 1 - Übersicht

- Audio-Signalweg
- Abtastung und Quantisierung
- PCM und WAV
- NumPy-Arrays
- Beispiele
- Zusammenfassung
- Übungsaufgaben zum 4.5.2026

Vom Schall zur Datei

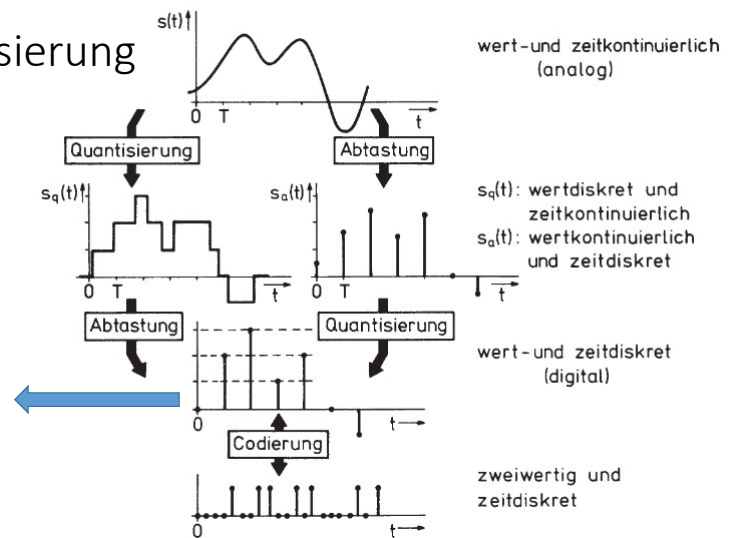
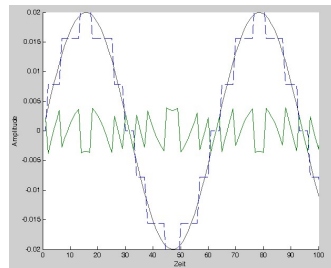
- Schall: kontinuierliches physikalisches Signal (Schalldruck, Schallpegel, Schallgeschwindigkeit)
- Mikrofon: Wandler von Schalldruck in analoge elektrische Spannung
- ADC (Analog-Digital-Converter): Abtastung und Quantisierung
- Audio-Interface/Soundkarte: digitale Repräsentation
- Python verarbeitet Wertefolgen

Abtastung und Quantisierung

- Abtastfrequenz (sample rate): Wie oft pro Sekunde wird das analoge Signal gemessen?
- Bittiefe (Auflösung, Wordbreite): Wie fein werden Amplitudenwerte unterschieden?
- Quantisierungsfehler
- Nyquist-Kriterium: Abtastrate muss größer sein als das Doppelte der höchsten relevanten Frequenz
 - zu niedrige Abtastfrequenz führt zu Aliasing

Abtastung und Quantisierung

8 bit



Quelle: Ohm, Lücke

Abtastung und Quantisierung

Die binäre Übertragung beliebiger analoger, also wert- und zeitkontinuierlicher Quellensignale ist nicht fehlerfrei möglich.

Für die aliasfreie Abtastung ist eine Bandbegrenzung notwendig.

Bei der Umwandlung der wertkontinuierlichen in wertdiskrete Signale ist eine Quantisierung notwendig.

In einem letzten Schritt wird die Information in eine Folge von Binärzahlen umgesetzt.

Diese Binärzahlen können dann mit Methoden der Digitaltechnik übertragen und verarbeitet werden.

[Ohm, Lücke]

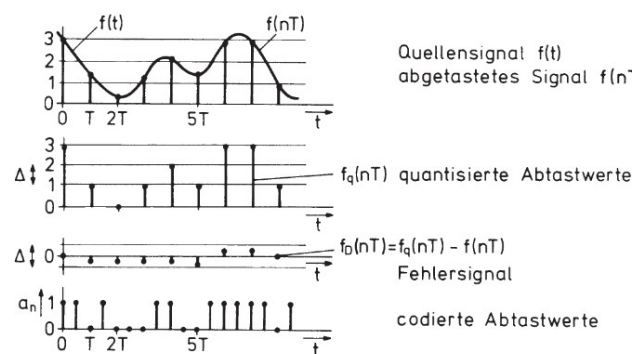
PCM und WAV

- PCM: direkte diskrete Repräsentation von Abtastwerten
- WAV: Containerformat
- Für den Einstieg wichtig:
 - Kanalzahl
 - Sample Width
 - Framerate / Samplerate
 - Anzahl Frames
- Vorteil: transparent, wenig „Magie“

PCM

1. Das als bandbegrenzt angenommene Quellensignal $f(t)$ wird unter Berücksichtigung des Abtasttheorems abgetastet.
2. Die Abtastwerte $f(nT)$ werden gerundet, im Bildbeispiel auf den nächstliegenden ganzzahligen Amplitudenwert $f_q(nT)$. Dieser Rundungsvorgang wird auch als Quantisierung bezeichnet, er kann nicht mehr rückgängig gemacht werden. Für jeden Abtastwert entsteht ein Rundungs- oder Quantisierungsfehler.
3. Die gerundeten Abtastwerte können in eine endliche Binärzahl umcodiert werden.
z.B. (üblich Audiotechnik: 16 Bit, 2^{16} Q-Stufen)

Quantisierungsstufe	Binärzahl
0	00
1	01
2	10
3	11



[Ohm, Lücke]

WAV

WAVE-Dateiformat:

- Containerformat zur digitalen Speicherung von Audiodaten
- Basiert auf Resource Interchange File Format (RIFF) (Microsoft, Windows)
- FORMAT-Abschnitt: enthält Parameter wie z. B. die Abtastrate.
- DATA-Abschnitt: enthält die PCM-Rohdaten, darf nicht vor dem FORMAT-Abschnitt stehen.

```
RIFF WAVE Chunk
groupID  = 'RIFF'
riffType = 'WAVE'
```

```
Format Chunk
ckID = 'fmt '
```

```
Sound Data Chunk
ckID = 'data'
```

[Wikipedia]

Aufgabe 1

Ordnen Sie die Begriffe in der richtige Reihenfolge an und markieren Sie, wo der Übergang von analog zu digital liegt:

- WAV-Datei
- Mikrofon
- Python-Array
- Schallereignis
- ADC / Audio-Interface / Soundkarte
- DAC / Audio-Interface / Soundkarte
- Lautsprecher / Kopfhörer

Zeit: 5 Minuten

NumPy-Arrays

- Audiodaten werden in `ndarray` eingelesen
- Mono: eindimensionales Array
- Stereo/Mehrkanal: zwei- oder mehrdimensionales Array
- Wichtige Eigenschaften:
 - `shape`
 - `type`
 - Zahlenbereich

ndarray ist ein generischer mehrdimensionaler Container; erlaubt mathematische Operationen auf ganzen Datenblöcken und verwendet dabei eine ähnliche Syntax wie bei ähnlichen Operationen zwischen skalaren Elementen.

Mit den Attributen `ndim` und `shape` können wir uns die Anzahl der Dimensionen und den Umriss von `array_2d` ausgeben lassen

Beispiel 1: Sinuston erzeugen

```
import numpy as np
from scipy.io.wavfile import write

sr = 44100                # Samples pro Sekunde
dauer = 1.0               # Sekunden
frequenz = 440            # Hz

t = np.linspace(0, dauer, int(sr * dauer), endpoint=False)
signal = 0.5 * np.sin(2 * np.pi * frequenz * t)

audio_int16 = (signal * 32767).astype(np.int16)
write("sinus_440Hz.wav", sr, audio_int16)
```

Aufgabe 2

Ändern Sie im Code:

- die Frequenz, z.B. auf 880 Hz
- die Dauer, z.B. auf 0,5 s
- die Abtastfrequenz (sample rate), z.B. auf 8000 Hz

Fragen:

- 1.a Wie ändert sich die Zahl der Abtastwerte (samples)?
- 1.b Welche Änderung betrifft die Tonhöhe, die Dateigröße, die Qualität?

Dauer: 7 Minuten

Beispiel 2: Stereo und Mono

```
import numpy as np
from scipy.io.wavfile import write, read

sr = 44100
dauer = 1.0
t = np.linspace(0, dauer, int(sr * dauer), endpoint=False)

links = 0.5 * np.sin(2 * np.pi * 440 * t)
rechts = 0.5 * np.sin(2 * np.pi * 660 * t)

stereo = np.column_stack((links, rechts))
write("stereo.wav", sr, (stereo * 32767).astype(np.int16))

sr2, stereo_data = read("stereo.wav")
mono = stereo_data.mean(axis=1).astype(np.int16)
write("mono_aus_stereo.wav", sr2, mono)

print("Stereo-Shape:", stereo_data.shape)
print("Mono-Shape:", mono.shape)
```

Aufgabe 3

Verändern Sie den Code so, dass

- rechts wesentlich leiser ist als links
- anschließend nur der linke Kanal gespeichert wird

3.a Welche `shapes` haben `stereo`, `stereo_data`, „nur der linke Kanal“?

3.b Warum ist „nur linker Kanal“ etwas anderes als „Mono-Mischung“?

Dauer: 7 Minuten

Zeitbereich und Frequenzbereich

- Zeitbereich: verändert sich der momentane Wert über die Zeit?
- Frequenzbereich: Welche Frequenzanteile sind enthalten?
- DFT: diskrete Fourier-Transformation
- FFT: Fast Fourier Transformation; effizienter Algorithmus zur Berechnung der DFT
- Für reelle Signale praktisch: `np.fft.rfft`

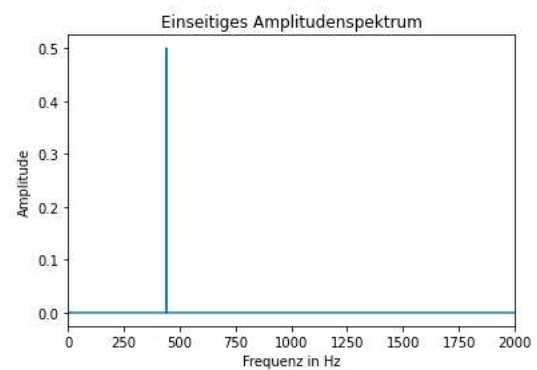
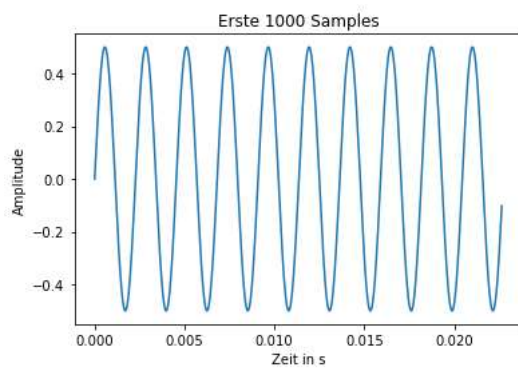
Beispiel 3: WAV-Datei einlesen und im Frequenzbereich betrachten

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io.wavfile import read

sr, data = read("sinus_440Hz.wav")
N = len(data)

spektrum = np.abs(np.fft.rfft(data))
frequenzen = np.fft.rfftfreq(len(data), d=1/sr)

# Einseitiges Amplitudenspektrum
amplitude = np.abs(spektrum) / N
amplitude[1:] *= 2
```



Kurzzeit-Fourier-Transformation

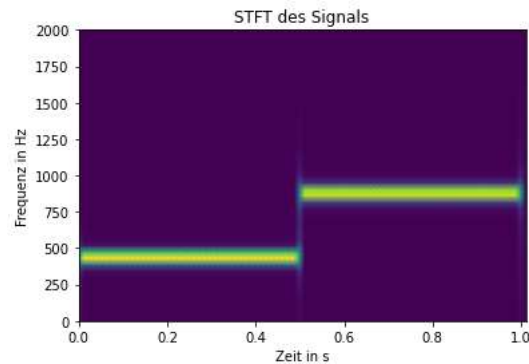
- FFT beschreibt das Signal insgesamt
- Aber: Was passiert, wenn sich das Signal im Verlauf ändert?
STFT = Fourier-Transformation auf kurzen Zeitfenstern
- sichtbar wird: Frequenz über Zeit

Beispiel 4: STFT berechnen und darstellen

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io.wavfile import write, read
from scipy.signal import stft

sr = 44100
dauer = 1.0
t = np.linspace(0, dauer, int(sr * dauer), endpoint=False)
# signal: zuerst 440 Hz, dann 880 Hz
signal = np.concatenate([
    0.5 * np.sin(2 * np.pi * 440 * t[:len(t)//2]),
    0.5 * np.sin(2 * np.pi * 880 * t[len(t)//2:])
])
write("signal.wav", sr, (signal * 32767).astype(np.int16))

sr2, signal_1 = read("stereo.wav")
f, tt, Zxx = stft(signal_1, fs=sr2, nperseg=1024)
```



Aufgabe 3

Arbeiten Sie mit Beispielcode 3 oder 4 und bearbeiten Sie eine der beiden Varianten:

- **FFT-Variante:** Erzeugen Sie statt 440 Hz einen Ton aus 440 Hz + 660 Hz und prüfen Sie, an welchen Stellen im Spektrum Peaks sichtbar werden.
- **STFT-Variante:** Verschieben Sie den Wechsel von 440 Hz zu 880 Hz Kanal auf 0,25 s.
- Fragen:
- Warum zeigt eine einzelne FFT beide Frequenzen, aber nicht den Wechselzeitpunkt?
- Warum kann die STFT den Wechselzeitpunkt sichtbar machen?

Dauer: 7 Minuten

Zusammenfassung

- Stereo = 2-D-Array
- STFT macht Frequenzänderungen über die Zeit sichtbar
- Zentrale Begriffe heute:
 - Sample rate
 - Bittiefe
 - PCM / WAV
 - `shape`, `dtype`
 - FFT / STFT
- LLMs im Kurs:
 - erlaubt als Werkzeug
 - nicht als Ersatz für Verstehen

Aufgabe 4

Sie möchten eine Sprachaufnahme für eine spätere Python-Analyse speichern.

Diskutieren Sie kurz:

- Mono oder Stereo?
- Abtastrate eher 8000, 16000 oder 44100 Hz?
- Wie groß ist jeweils die Datenrate?
- Wieviel Speicherplatz benötigen Sie für die Aufnahme?

Dauer: 7 Minuten

Übungsaufgabe 1 – Abgabe bis zum 3.6.2026 (moodle)

Concept Map zum Audio-Signalweg

Erstellen Sie auf einer Seite eine Concept Map (<https://de.wikipedia.org/wiki/Concept-Map>) zu:

- Schall
- Mikrofon
- ADC
- Samplerate
- Bittiefe
- PCM
- WAV
- Kanalzahl
- NumPy-Array

a. Definieren Sie jeden Begriff in 1–2 Sätzen.

b. Verbinden Sie die Begriffe mit Pfeilen.

c. Markieren Sie drei Begriffe, die zur **realen Welt / Hardware** gehören, und drei, die zur **digitalen Repräsentation in Python** gehören.

Übungsaufgabe 2 - Abgabe bis zum 3.6.2026 (moodle)

Audiosignale einlesen, bearbeiten, anzeigen

Schreiben Sie ein kurzes Python-Skript, das:

- ein Sprachsignal über das Mikrofon einliest und
- es als WAV speichert,
- das Signal aus der WAV-Datei wieder einliest, und
 - a. die ersten 20 ms plottet,
 - b. zusätzlich eine zweite Version dieser Sprachaufnahme mit veränderter Samplerate speichert.

Abgabe

- Python-Datei
- zwei erzeugte WAV-Dateien
- Rechnung: Wie viel Speicherplatz benötigt die WAV-Datei (ca.)?
- 5 Sätze Reflexion:
 - Was ist im Plot sichtbar?
 - Welche Rolle spielt die Samplerate?
 - Stimmt Ihre Rechnung?

weiter Übungsaufgabe 2

Hilfe

Anzeigen der vorhandenen Audiogeräte mit

```
import sounddevice as sd
print(sd.query_devices())
```

Aufnahme (Beispiel: wenn device = 2):

```
aufnahme =sd.rec(int(dauer * samplerate),
                  samplerate=samplerate,
                  channels=1,
                  dtype="int16",
                  device=2)
```

Quellen zu Thema 1

- Numpy ndarray <https://www.python4data.science/de/latest/workspace/numpy/ndarray.html>
- NumPy FFT: rfft, rfftfreq <https://numpy.org/doc/stable/reference/generated/numpy.fft.fft.html>
- SciPy STFT: `scipy.signal.stft`
- Python Standard Library Wave: `wave` <https://docs.python.org/3/library/wave.html>
- SciPy WAVFILE: `wavfile.read` und `wavfile.write`
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.wavfile.read.html>
- NI: Digitizer, Sample Rate, Nyquist, Aliasing https://www.ni.com/de/shop/data-acquisition/measurement-fundamentals/analog-fundamentals/acquiring-an-analog-signal--bandwidth--nyquist-sampling-theorem-.html?srsltid=AfmBOopY6Mh_zJ1Iro_fZGlvhvXizuhHJ6nyvzOjsNbM_u6ED7HCx5

Für sehr Interessierte:

- Signal Processing `scipy.signal` https://docs.scipy.org/doc/scipy/tutorial/signal.html?utm_source=chatgpt.com
- Ohm, J.-R., Lüke, H. D.: Signalübertragung. Springer-Verlag 2014 <https://link.springer.com/content/pdf/10.1007/978-3-642-53901-5.pdf>